Reinforcement Learning | Markov Decision Processes

Selene · October 13, 2026

1 Markov Processes

The Markov property states that the future state depends only on the current state, not on the past history.

We can use a state-transition matrix P to describe the transition probabilities $p(s_{t+1} = s' \mid s_t = s)$:

$$P = \begin{pmatrix} p(s_1 \mid s_1) & p(s_2 \mid s_1) & \cdots & p(s_N \mid s_1) \\ p(s_1 \mid s_2) & p(s_2 \mid s_2) & \cdots & p(s_N \mid s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(s_1 \mid s_N) & p(s_2 \mid s_N) & \cdots & p(s_N \mid s_N) \end{pmatrix}.$$

2 Markov Reward Processes

A Markov reward process adds a reward function to a Markov chain. The reward function gives the expected one-step reward. For an episode with a finite number of steps, the **return** is the discounted sum of rewards. If the rewards after time t are $r_{t+1}, r_{t+2}, r_{t+3}, \ldots$, then the return is

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots + \gamma^{T-t-1} r_T.$$

Here, T is the final time step and γ is the discount factor. When $0 < \gamma < 1$, rewards further in the future are discounted more heavily, expressing a preference for rewards received sooner. We define the **state-value function** as the expected return:

$$\begin{aligned} V^t(s) &= \mathbb{E}[G_t \mid s_t = s] \\ &= \mathbb{E}\left[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots + \gamma^{T-t-1} r_T \mid s_t = s\right]. \end{aligned}$$

The superscript t records the dependence on time in a finite-horizon episode.

3 The Bellman Equation

The Bellman equation decomposes the value of a state into two parts: the **expected reward for the next step**, plus the **discounted expected value of the next state**.

For a time-homogeneous process with an infinite discounted horizon, we write the stationary value function as $V(s)$:

$$V(s) = R(s) + \gamma \sum_{s'} p(s' \mid s) V(s').$$

Here, $R(s)$ is the expected one-step reward from state s, and $p(s' \mid s)$ is the probability of moving from s to s'. The term $\gamma \sum_{s'} p(s' \mid s) V(s')$ represents the expected discounted return after the first step. For a finite horizon, the corresponding recursion uses V^t on the left and V^{t+1} on the right, with $V^T(s) = 0$.

We can write the Bellman equation in matrix form:

$$\begin{pmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{pmatrix} = \begin{pmatrix} R(s_1) \\ R(s_2) \\ \vdots \\ R(s_N) \end{pmatrix} + \gamma \begin{pmatrix} p(s_1 \mid s_1) & p(s_2 \mid s_1) & \cdots & p(s_N \mid s_1) \\ p(s_1 \mid s_2) & p(s_2 \mid s_2) & \cdots & p(s_N \mid s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(s_1 \mid s_N) & p(s_2 \mid s_N) & \cdots & p(s_N \mid s_N) \end{pmatrix} \begin{pmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{pmatrix}.$$

Combining the Bellman equations for all states gives $V = R + \gamma P V$. Rearranging yields the **analytical solution**:

$$V = (I - \gamma P)^{-1} R.$$

For a finite state space and $0 \leq \gamma < 1$, the matrix $I - \gamma P$ is invertible.

4 Computing State Values

- **Monte Carlo:** Starting from state s, sample multiple complete trajectories, compute the return of each trajectory, and average those returns.
- **Dynamic programming:** When the rewards and transition probabilities are known, repeatedly apply the Bellman equation to update the values until they converge. Updating a state's value using the current estimates of its successor states is called **bootstrapping**.

5 MDPs and MRPs

In an MRP, the next state is sampled directly from the transition distribution over the state space S. In an MDP, an action comes first: the agent chooses an action a, and the next state is then sampled according to the transition distribution conditioned on both the state and the action. A policy $\pi(a \mid s)$ describes how the agent chooses actions in state s.



